Database Normalization Cheat Sheet

What is Normalization?

The process of organizing database tables to minimize redundancy and ensure data integrity.

Normal Forms Summary

Form	Goal	Rule	Key Issue Fixed
1NF	Atomic Values	No repeating groups; each cell has one value	Redundant data
2NF	Full Dependency	Must be in 1NF + no partial dependencies	Composite key anomalies
3NF	No Transitive	Must be in 2NF + non-keys don't depend on non-keys	Chain dependencies
BCNF	Superkey Focus	Every determinant must be a candidate key	Complex candidate keys

---

1NF: First Normal Form

Rule: Remove repeating groups. Every field must contain atomic (single, indivisible) values.

Example:

✖ **Not 1NF** (Repeating groups)
StudentCourses Table
StudentID | Name | Courses
101 | Alice | Math, Physics, Chemistry
102 | Bob | Biology, Chemistry

✅ **1NF** (Atomic values)
StudentCourses Table
StudentID | Name | Course
101 | Alice | Math
101 | Alice | Physics
101 | Alice | Chemistry

102 | Bob | Biology
102 | Bob | Chemistry

2NF: Second Normal Form

Rule: Must be in 1NF + remove partial dependencies. Every non-key attribute must depend on the **entire primary key**, not just part of it.

Example:

✖ **Not 2NF** (Partial dependency on composite key)
StudentCourses Table (Primary Key: StudentID + CourseID)
StudentID | CourseID | CourseName | Instructor
101 | C1 | Math | Dr. Smith
101 | C2 | Physics | Dr. Jones
102 | C1 | Math | Dr. Smith
Issue: CourseName and Instructor depend only on CourseID, not the full key

✅ **2NF** (Full dependency)
StudentCourses Table
StudentID | CourseID
101 | C1
101 | C2
102 | C1

Courses Table
CourseID | CourseName | Instructor
C1 | Math | Dr. Smith
C2 | Physics | Dr. Jones

3NF: Third Normal Form

Rule: Must be in 2NF + remove transitive dependencies. Non-key attributes must depend **only on the primary key**, not on other non-key attributes.

Example:

✖ **Not 3NF** (Transitive dependency)
Orders Table (Primary Key: OrderID)
OrderID | CustomerID | CustomerName | CustomerCity
1001 | C1 | Alice | New York
1002 | C2 | Bob | London
1003 | C1 | Alice | New York
Issue: CustomerName and CustomerCity depend on CustomerID, not OrderID

✅ **3NF** (No transitive dependency)
Orders Table
OrderID | CustomerID
1001 | C1
1002 | C2
1003 | C1

Customers Table
CustomerID | CustomerName | CustomerCity
C1 | Alice | New York
C2 | Bob | London

BCNF: Boyce-Codd Normal Form

Rule: A stricter version of 3NF. For every non-trivial functional dependency X → Y, **X must be a superkey** (candidate key).

When to Use: Complex databases with multiple candidate keys.

Example:

✕ **Not BCNF** (Non-key determinant)
Professor_Course Table
ProfessorID | CourseID | Time
P1 | C1 | 9:00 AM
P1 | C2 | 10:00 AM
P2 | C1 | 9:00 AM
Issue: CourseID determines Time, but CourseID is not a superkey

✓ **BCNF** (Decomposed)
Professor_Course Table
ProfessorID | CourseID
P1 | C1
P1 | C2
P2 | C1

Course_Time Table
CourseID | Time
C1 | 9:00 AM
C2 | 10:00 AM

Quick Comparison

Normalization Benefits:

- ✓ Reduces redundancy
- ✓ Improves data integrity
- ✓ Easier to update/maintain
- ✓ Reduces storage space

Denormalization Trade-offs:

- Faster read queries (fewer joins)
- More storage needed
- Higher update/insert complexity
- Better for analytics & reporting

Normalization Checklist

- [ ] Is every column atomic? → 1NF
- [ ] Does every non-key attribute depend on the entire primary key? → 2NF
- [ ] Do non-key attributes only depend on the primary key? → 3NF
- [ ] Is every determinant a superkey? → BCNF

Key Terms

- **Atomic:** Single, indivisible value
- **Functional Dependency:** Attribute A determines attribute B
- **Candidate Key:** Minimal set of attributes that uniquely identifies a row
- **Partial Dependency:** Non-key attribute depends on part of a composite primary key
- **Transitive Dependency:** Non-key attribute depends on another non-key attribute
- **Superkey:** Any set of attributes that uniquely identifies a row (includes candidate keys)

When to Normalize vs. Denormalize

Normalize when:

- Data integrity is critical
- Frequent updates/inserts
- Complex relationships

Denormalize when:

- Read performance is critical
- Reporting/analytics focus
- Historical/read-only data

For most applications, **3NF is sufficient** and provides good balance between integrity and performance.